



南開大學

Nankai University

计算机学院

网络技术与应用实验报告

实验 3：通过编程获取 IP 地址与 MAC 地址的对应关系

姓名：王旭

学号：2312166

专业：计算机科学与技术

2025 年 11 月 23 日

目录

1 实验二中两个关键函数的原理回顾	2
1.1 NPcap 的设备列表获取方法	2
1.1.1 参数说明:	2
1.1.2 相关的关键数据结构:	2
1.1.3 NPcap 设备列表获取演示:	3
1.2 NPcap 的网卡设备打开方法	3
1.2.1 参数说明:	3
1.2.2 NPcap 网卡设备打开演示:	4
2 NPcap 的数据包发送方法	4
2.1 参数说明:	4
2.2 验证 pcap_sendpacket() 函数的发送功能:	5
2.3 演示程序:	5
2.3.1 网卡设备选择的考虑:	6
2.4 运行结果:	7
3 ARP 协议原理	7
3.1 ARP 的背景与设计动机	7
3.2 ARP 的标准工作流程示例	8
3.3 ARP 数据包结构	8
4 获取 IP 地址与 MAC 地址的映射的程序设计	9
4.1 局域网选择的考虑	9
4.2 最终程序设计	10
4.2.1 引入头文件和库	10
4.2.2 定义以太网帧和 ARP 报文结构	10
4.2.3 定义全局变量, 用于回调函数通信	11
4.2.4 数据包处理回调函数 packet_handler	11
4.2.5 IP 地址解析辅助函数 parse_ip	11
4.2.6 main 主函数	12
5 程序运行结果	14
5.1 通过手机热点查看手机与电脑的 IP 与 MAC 地址	14
5.2 程序运行的 MAC 地址结果	15

1 实验二中两个关键函数的原理回顾

1.1 Npcap 的设备列表获取方法

设备列表获取是 Npcap 编程的第一步，它的核心目的是让程序能够发现系统中所有可用的网络接口。在 Windows 系统中，一个计算机可能有多个网络接口，比如有线网卡、无线网卡、虚拟网卡等。Npcap 通过特定的函数来枚举这些接口，并为每个接口提供详细的信息，方便我们选择要监控的目标。

Npcap 中用于获取设备列表的核心函数 `pcap_findalldevs`

```
1 int pcap_findalldevs(pcap_if_t **alldevsp, char *errbuf);
```

1.1.1 参数说明:

1. **alldevsp**: 这是一个指向设备链表头指针的指针，函数执行成功后，会通过这个参数返回设备链表的头指针
2. **errbuf**: 错误信息缓冲区，如果函数执行失败，会在这里存储错误描述
3. **返回值**: 成功返回 0，失败返回-1

1.1.2 相关的关键数据结构:

`pcap_if_t` 结构体（设备信息结构）

```
1 struct pcap_if {
2     struct pcap_if *next;           // 指向下一个设备的指针
3     char *name;                     // 设备名称（用于打开设备）
4     char *description;              // 设备描述（人类可读的描述）
5     struct pcap_addr *addresses;    // 设备地址列表
6     u_int flags;                    // 设备标志位
7 };
```

`pcap_addr_t` 结构体（地址信息结构）

```
1 struct pcap_addr {
2     struct pcap_addr *next;         // 下一个地址
3     struct sockaddr *addr;          // IP地址
4     struct sockaddr *netmask;       // 子网掩码
5     struct sockaddr *broadaddr;     // 广播地址
6     struct sockaddr *dstaddr;       // 目标地址（点对点连接）
7 };
```

库函数会构建一个 `pcap_if_t` 结构的链表。链表中的每个节点代表一个网络接口，包含了该接口的所有相关信息。这个链表采用单向链表的形式组织，通过 `next` 指针连接各个设备。`pcap_findalldevs()` 函数会将链表的头指针通过参数返回，调用者就可以通过遍历这个链表来查看所有的可用设备。

1.1.3 Npcap 设备列表获取演示:

```

Npcap设备列表获取演示
=====
1. \Device\NPF_{910F7DAB-50B3-4852-8704-2A74FC8A1661} - WAN Miniport (Network Monitor)
2. \Device\NPF_{6526F4B3-23F6-4839-BA2A-450E6A845412} - WAN Miniport (IPv6)
3. \Device\NPF_{22D86905-76CA-446A-8F57-DF5B598162FD} - WAN Miniport (IP)
4. \Device\NPF_{0A887C94-9477-4075-8593-978E4211744E} - Realtek 8821CE Wireless LAN 802.11ac PCI-E NIC
5. \Device\NPF_{B114056E-1B81-47FE-8230-EBC2A02752C4} - VMware Virtual Ethernet Adapter for VMnet8
6. \Device\NPF_{8005DF67-9681-404D-9710-DAF67A5A4025} - VMware Virtual Ethernet Adapter for VMnet1
7. \Device\NPF_{D9D7DE5D-7F32-4366-B926-E7BC13BD3530} - Microsoft Wi-Fi Direct Virtual Adapter #2
8. \Device\NPF_{56D1FF55-2E15-49DC-8D8D-0850383CA1FC} - Microsoft Wi-Fi Direct Virtual Adapter
9. \Device\NPF_{34D905C6-7189-4DC1-B4F6-1195B36AF8C6} - Hyper-V Virtual Ethernet Adapter #2
10. \Device\NPF_{040450B3-CD87-4E53-AB8D-668AE1BF8C0F} - Hyper-V Virtual Ethernet Adapter
11. \Device\NPF_{Loopback} - Adapter for loopback traffic capture
12. \Device\NPF_{143C9220-8B62-4871-A20C-823A88F19746} - TAP-Windows Adapter V9
13. \Device\NPF_{C9BC77C9-69B2-462F-A2D8-EA367CAEDF68} - Wintun Userspace Tunnel
共找到 13 个设备
请按任意键继续...
    
```

图 1.1: Npcap 设备列表获取演示

可以看到 Npcap 成功检测到系统中的所有网络接口，共找到 13 个网络设备，这包括物理网卡、虚拟网卡、VPN 适配器等。

1.2 Npcap 的网卡设备打开方法

在成功获取设备列表后，下一步就是打开选定的网络接口设备。打开设备是 Npcap 编程中的关键步骤，它建立了应用程序与网络接口之间的通信通道。这个过程涉及到设备句柄的创建、工作模式的设置以及必要的参数配置。

Npcap 中用于网卡设备打开的函数 `pcap_open_live`

```

1 pcap_t* pcap_open_live(const char* device, int snaplen, int promisc, int to_ms, char*
    errbuf);
    
```

1.2.1 参数说明:

1. **device:** 要打开的设备名称字符串, 这个名称来自 `pcap_findalldevs()` 获取的设备列表中的 `name` 字段。
2. **snaplen:** 指定每个数据包要捕获的最大长度 (字节数)。
3. **promisc:** 是否启用混杂模式, 0 表示非混杂模式, 只捕获发给本机的数据包; 1 表示混杂模式, 捕获所有流经网络接口的数据包
4. **to_ms:** 指定在捕获数据包时的超时时间 (ms), 设置为 0 表示无超时 (阻塞模式)。
5. **errbuf:** 错误信息缓冲区, 用于接收错误描述的字符数组
6. **返回值:** 成功: 返回 `pcap_t*` 类型的设备句柄指针; 失败则返回 `NULL`, 错误信息存储在 `errbuf` 中

当调用 `pcap_open_live()` 函数时, Npcap 在底层执行一系列复杂的操作。首先, 函数会验证传入的设备名称是否有效, 检查该设备是否确实存在于系统中并且支持数据包捕获功能。这个验证过程涉及到与 Npcap 驱动程序的通信。

验证通过后，Npcap 会创建一个 pcap_t 结构体实例，这个结构体是 Npcap 的核心数据结构，它包含了设备的所有状态信息、配置参数以及各种回调函数指针。创建这个结构体相当于为后续的数据包捕获操作准备了完整的工作环境。

接下来，函数会根据传入的参数配置设备的工作模式。其中最重要的就是混杂模式的设置。在非混杂模式下，网络接口卡只会接收目的地为本机的数据包；而在混杂模式下，网卡会接收所有流经网络的数据包，这对于网络监控和分析至关重要。这个模式的设置需要操作系统层面的权限配合。

同时，函数还会设置数据包的截断长度和超时时间的设置。

最后，函数会初始化各种内部缓冲区和结构，为实际的数据包捕获做好准备。整个过程完成后，返回的设备句柄就成为了后续所有数据包操作的基础。

1.2.2 Npcap 网卡设备打开演示:

```

=====
Npcap网卡设备打开方法演示
=====
可用设备列表:
1. \Device\NPF_{910F7D4B-5083-4852-8704-2A7AFC8A1661} - WAN Miniport (Network Monitor)
2. \Device\NPF_{6526F4B3-23F6-4839-B42A-458E608A5412} - WAN Miniport (IPv6)
3. \Device\NPF_{22D86906-76CA-446A-8F57-D65B59B162FD} - WAN Miniport (IP)
4. \Device\NPF_{0A887C94-9477-4075-8593-978E421174AE} - Realtek 8821CE Wireless LAN 802.11ac PCI-E NIC
5. \Device\NPF_{B114056E-1B81-47FE-8238-EBC2A02752CA} - VMware Virtual Ethernet Adapter for VMnet8
6. \Device\NPF_{80050F67-9681-404D-9710-DAF67A5A4025} - VMware Virtual Ethernet Adapter for VMnet1
7. \Device\NPF_{D907DE5D-7F32-4366-B926-E7BC13B03530} - Microsoft Wi-Fi Direct Virtual Adapter #2
8. \Device\NPF_{56D1FF55-2E15-49DC-808D-0850383CA1FC} - Microsoft Wi-Fi Direct Virtual Adapter
9. \Device\NPF_{34D995C6-7189-4DC1-B4F6-1195B36AF8C6} - Hyper-V Virtual Ethernet Adapter #2
10. \Device\NPF_{040450B3-CD87-4E53-A88D-668AE1BF8C0F} - Hyper-V Virtual Ethernet Adapter
11. \Device\NPF_{loopback} - Adapter for loopback traffic capture
12. \Device\NPF_{143C9220-8B62-4871-A20C-823A8BF19746} - TAP-Windows Adapter V9
13. \Device\NPF_{C98C77C9-69B2-462F-A2D8-EA367CAEDF68} - Wintun Userspace Tunnel
共找到 13 个设备
选择要打开的设备 (输入设备编号) : 4

尝试打开设备4: Realtek 8821CE Wireless LAN 802.11ac PCI-E NIC
设备打开成功!
设备句柄: 0x1a4e0b0a0f0
设备已关闭
程序结束
请按任意键继续. . .
    
```

图 1.2: Npcap 网卡设备打开演示

2 Npcap 的数据包发送方法

在 IP 数据报捕获与分析编程实验的基础上，我们学习了 Npcap 的设备列表获取和网卡设备打开的具体方法并进行了具体的代码实践工作，现在我们本实验目标的第一步在于学习 Npcap 的数据包发送方法。

Npcap 中用于数据包发送方法的核心函数 pcap_sendpacket

```

1 int pcap_sendpacket(pcap_t *p, const u_char *buf, int size);
    
```

2.1 参数说明:

1. **p:** 已打开的设备句柄 (来自 pcap_open_live)
2. **buf:** 指向要发送的原始字节数据 (整个以太网帧)
3. **size:** 数据长度 (字节数)
4. **返回值:** 0 表示成功, -1 表示失败

2.2 验证 pcap_sendpacket() 函数的发送功能:

为了验证发送功能,我们先不直接构造复杂的 ARP,而是先发送一个最简单的伪造以太网帧,等到后面我们在进行第二步获取 IP 地址与 MAC 地址的映射关系时,我们再对 ARP 进行详细的说明与构造,并对其进行 pcap_sendpacket() 的函数调用。

ethernet_header 结构体 (以太网帧头结构)

```
1 // 以太网帧头结构
2 struct ethernet_header {
3     u_char dest_mac[6];    // 目的 MAC
4     u_char src_mac[6];     // 源 MAC
5     u_short ether_type;    // 类型
6 };
```

我们构造下面的这样一个原始以太网帧数据包:

- **目的 MAC:** 广播地址 FF:FF:FF:FF:FF:FF
- **源 MAC:** 构造一个伪造的地址 (不属于任何真实设备), 比如 11:22:33:44:55:66
- **类型字段:** 0x0800 (表示后面是 IPv4, 虽然我们暂时不填 IP 内容)
- **载荷:** 一个字符串, 比如"Hello"

2.3 演示程序:

NPcap 数据包发送方式的演示程序 send_test.cpp

```
1 #include <iostream>
2 #include <cstring>
3 #include "pcap.h"
4
5 // 以太网帧头结构
6 struct ethernet_header {
7     u_char dest_mac[6];    // 目的 MAC
8     u_char src_mac[6];     // 源 MAC
9     u_short ether_type;    // 类型
10 };
11 int main() {
12     // 1. 获取设备列表
13     // 2. 显示设备列表
14     // 3. 选择设备
15     // 4. 打开设备 (混杂模式, 超时1秒)
16     device_handle = pcap_open_live(device->name, 65536, 1, 1000, errbuf);
17     .....
18     // 5. 构造一个简单的以太网帧
19     u_char packet[64]; // 最小以太网帧为64字节
20     memset(packet, 0, sizeof(packet)); // 先清零
21
22     ethernet_header* eth = (ethernet_header*)packet;
```

```

23
24 // 设置目的MAC为广播地址 FF:FF:FF:FF:FF:FF
25 memset(eth->dest_mac, 0xFF, 6);
26
27 // 设置源MAC为伪造地址 11:22:33:44:55:66
28 eth->src_mac[0] = 0x11;
29 eth->src_mac[1] = 0x22;
30 eth->src_mac[2] = 0x33;
31 eth->src_mac[3] = 0x44;
32 eth->src_mac[4] = 0x55;
33 eth->src_mac[5] = 0x66;
34
35 // 设置类型为IPv4 (0x0800)
36 eth->ether_type = htons(0x0800);
37
38 // 在后面加上载荷数据
39 const char* payload = "Npcap Send Test";
40 memcpy(packet + sizeof(ethernet_header), payload, strlen(payload));
41
42 // 6. 发送数据包
43 if (pcap_sendpacket(device_handle, packet, sizeof(packet)) == 0) {
44     std::cout << "\n数据包发送成功!" << std::endl;
45     std::cout << "已发送一个伪造的以太网帧 (目的MAC=FF:FF:FF:FF:FF:FF(广播),
46     源MAC=11:22:33:44:55:66) " << std::endl;
47 } else {
48     std::cout << "\n发送失败: " << pcap_geterr(device_handle) << std::endl;
49 }
50 .....
51 }

```

我们略去在实验二中已经详细掌握的获取设备列表与打开具体网卡设备部分的代码，主要展示我们本实验中的构造原始以太网帧数据包 eth，以及通过 pcap_sendpacket 的方式来发送这个原始数据包的代码部分。

代码详见：Npcap 数据包发送方式的演示程序 send_test.cpp

2.3.1 网卡设备选择的考虑:

注意到我们选择的设备是第四个设备 Hyper-V 虚拟网卡:

```

1 4. \Device\NPF_{F3415588-79FA-42DB-B16D-9A6ED208DB26} - Hyper-V Virtual Ethernet
   Adapter #2

```

而不是我们在上个实验中用于捕获数据包的 Wi-Fi 无线网卡:

```

1 6. \Device\NPF_{0A887C94-9477-4075-8593-978E4211744E} - Realtek 8821CE Wireless LAN
   802.11ac PCI-E NIC

```

这是因为，我们 Npcap 的 pcap_sendpacket() 函数在 Wi-Fi 上有一定的限制:

- 只能发送某些类型的合法数据包 (如 DHCP、ARP)

- 不能发送任意伪造的以太网帧（出于安全和协议规范限制）
- 很多 Wi-Fi 驱动会直接拒绝这类操作，返回错误 31 (ERROR_GEN_FAILURE)

而虚拟网卡（如 Hyper-V）：是纯软件模拟的以太网设备，完全兼容以太网帧格式，无硬件限制，因此支持任意帧发送，也同样支持我们构造的这个原始以太网帧数据包的发送。虽然我们构造的这个数据包不合法（因为它缺少真正的 IP 头），但：它符合以太网帧的基本格式，并且我们的这个 Hyper-V 虚拟网卡驱动只检查格式，不检查内容合法性，因此在选择 Hyper-V 虚拟网卡设备时，它能被成功发送出去。

2.4 运行结果:

我们采用如下命令进行编译运行：

编译运行指令

```
1 # 编译
2 g++ src/send_test.cpp -o bin/send_test.exe -I Include -L Lib/x64 -lPacket -lwpcap
   -lws2_32 -liphlpapi -fexec-charset=UTF-8 -finput-charset=UTF-8
3 # 运行
4 ./bin/send_test.exe
```

```
=== Npcap 数据包发送测试 ===
1. \Device\NPF_{910F7DAB-50B3-4852-8704-2A74FC8A1661} - WAN Miniport (Network Monitor)
2. \Device\NPF_{6526F4B3-23F6-4839-BA2A-450E6A845412} - WAN Miniport (IPv6)
3. \Device\NPF_{22D86905-76CA-446A-8F57-DF5B59B162FD} - WAN Miniport (IP)
4. \Device\NPF_{F3415588-79FA-42D8-B16D-9A6ED208D826} - Hyper-V Virtual Ethernet Adapter #2
5. \Device\NPF_{A61680AB-7DE1-443B-8DE7-1BF08098ECC6} - Hyper-V Virtual Ethernet Adapter
6. \Device\NPF_{0A887C94-9477-4075-8593-978E4211744E} - Realtek 8821CE Wireless LAN 802.11ac PCI-E NIC
7. \Device\NPF_{B11A056E-1BB1-47FE-8230-EBC2A02752C4} - VMware Virtual Ethernet Adapter for VMnet8
8. \Device\NPF_{8005DE67-9681-404D-9710-DAF67A5A4025} - VMware Virtual Ethernet Adapter for VMnet1
9. \Device\NPF_{D9D7DE5D-7F32-4366-B926-E7BC13BD3530} - Microsoft Wi-Fi Direct Virtual Adapter #2
10. \Device\NPF_{56D1FF55-2E15-49DC-8D0D-0850383CA1FC} - Microsoft Wi-Fi Direct Virtual Adapter
11. \Device\NPF_{Loopback} - Adapter for loopback traffic capture
12. \Device\NPF_{143C9220-8862-4871-A20C-823A8BF19746} - TAP-Windows Adapter V9
13. \Device\NPF_{C9BC77C9-69B2-462F-A2D8-EA367CAEDF68} - Wintun Userspace Tunnel
共找到 13 个设备
请选择要使用的设备编号 (1-13): 4
设备打开成功，准备发送测试包...

数据包发送成功！
已发送一个伪造的以太网帧（目的MAC=FF:FF:FF:FF:FF:FF(广播)，源MAC=11:22:33:44:55:66）
```

图 2.3: Npcap 数据包发送演示

可以看到程序获取设备列表并正确打开了 Hyper-V 虚拟网卡，然后构造了我们目标合法的以太网帧，并调用 `pcap_sendpacket()` 成功将原始数据包发送到网络接口，最后操作系统和 Npcap 驱动接受了这个请求并执行了发送工作。

3 ARP 协议原理

3.1 ARP 的背景与设计动机

ARP (Address Resolution Protocol) 是一个地址解析协议：

在 *TCP/IP* 协议栈中，网络通信依赖两个关键地址：

- **IP 地址 (Logical Address)**：用于在网络层标识主机，支持跨网络路由。
- **MAC 地址 (Physical Address)**：用于在数据链路层标识网卡，是局域网内帧传输的物理依据。

然而，IP 地址无法直接用于局域网内的数据帧传输。以太网交换机仅根据 MAC 地址转发帧，而不识别 IP 地址。因此，在同一局域网内，发送方必须知道目标设备的 MAC 地址，才能构造有效的以太网帧。

因此，我们需要的 ARP 协议的核心目的就是：
在已知目标设备 IP 地址的前提下，动态解析出对应 MAC 地址，从而实现局域网内的二层通信。

3.2 ARP 的标准工作流程示例

假设主机 A(IP: 192.168.1.100 , MAC: AA:AA:AA:AA:AA:AA)希望与主机 B(IP: 192.168.1.101)通信：

1. 步骤 1：查询本地 ARP 缓存

主机 A 首先检查本地 ARP 缓存表(可通过命令 `arp -a` 查看)。若存在条目 192.168.1.101 -> BB:BB:BB:BB:BB:BB ，则直接使用，跳过后续步骤。

2. 步骤 2：广播 ARP 请求 (ARP Request)

若缓存未命中，主机 A 构造一个 ARP 请求报文，内容为：

- ” 谁的 IP 地址是 192.168.1.101 ？ 请回复你的 MAC 地址。 ”

该报文封装在以太网帧中，目的 MAC 设为广播地址 FF:FF:FF:FF:FF:FF ，确保局域网内所有设备都能收到。

3. 步骤 3：目标主机响应 (ARP Reply)

主机 B 收到请求后，发现目标 IP 与自身匹配。主机 B 构造一个 ARP 响应报文，内容为：

- ” 本主机的 IP 地址为 192.168.1.101 ， 我的 MAC 地址是 BB:BB:BB:BB:BB:BB 。”

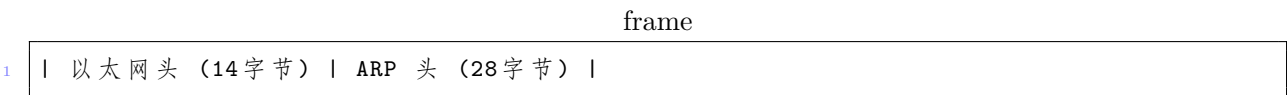
该响应以单播方式发送给主机 A （目的 MAC = AA:AA:AA:AA:AA:AA ）。

4. 步骤 4：更新缓存并通信

主机 A 收到响应后，将 (192.168.1.101 , BB:BB:BB:BB:BB:BB) 存入 ARP 缓存。后续通信可直接使用该 MAC 地址构造以太网帧。

3.3 ARP 数据包结构

ARP 包需要封装在以太网帧中，它的整体结构如下：



ethernet_header 结构体 (以太网帧头结构)

1 // 以太网帧头结构
2 struct ethernet_header {
3 u_char dest_mac[6]; // 目的 MAC
4 u_char src_mac[6]; // 源 MAC

```
5     u_short ether_type;    // 类型
6 };
```

arp_header 结构体 (ARP 头结构)

```
1 // ARP 头结构
2 struct arp_header {
3     u_int16_t htype;      // 硬件类型: 1 = 以太网
4     u_int16_t ptype;      // 协议类型: 0x0800 = IPv4
5     u_char hlen;          // 硬件地址长度: 6 (MAC 是 6 字节)
6     u_char plen;          // 协议地址长度: 4 (IPv4 是 4 字节)
7     u_int16_t oper;       // 操作码: 1=请求, 2=响应
8     u_char sender_mac[6]; // 发送方 MAC
9     u_char sender_ip[4];  // 发送方 IP
10    u_char target_mac[6];  // 目标 MAC (请求时填 0)
11    u_char target_ip[4];   // 目标 IP
12};
```

因此,我们在构建 APR 数据包时,首先将 ethernet_header 与 arp_header 拼接成一个 42 字节的缓冲区,然后按找上述 ARP 数据包的具体语义规则填入本机 MAC/IP 和目标 IP,最后调用 pcap_sendpacket(handle, buffer, 42) 就可以成功通过 Npcap 发送我们构建的 ARP 数据包了。

4 获取 IP 地址与 MAC 地址的映射的程序设计

4.1 局域网选择的考虑

在进行程序设计的过程中,我初始时想通过在同一 **校园网** 下的电脑和手机进行 ARP 测试,比如我们可以在电脑上查看我们连接校园网后电脑的 IP 地址和 MAC 地址,也可以在手机上查看到连接校园网后手机的 IP 地址和 MAC 地址,这样通过在电脑上想手机的 IP 地址发送 ARP 查询请求,通过我们写的 ARP 程序得到手机的 MAC 地址,然后再与我们手机实际的 ARP 进行对照,就能验证程序是否正确,但是我们在操作的过程中发现, **我们的 ARP 请求无法到达手机,手机也无法收到广播包。**

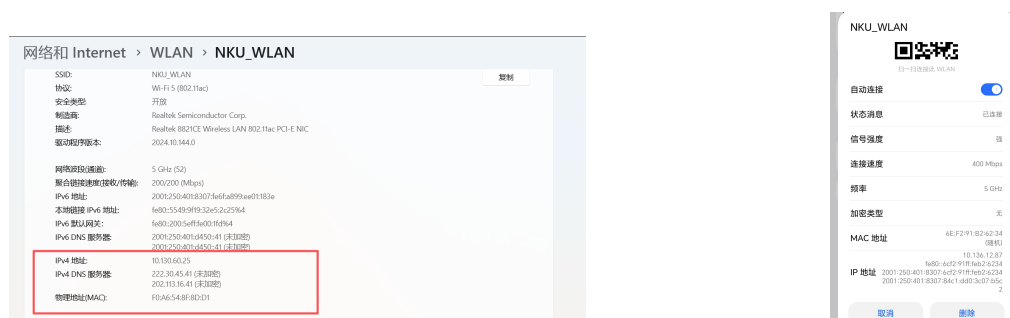


图 4.4: 校园网下电脑和手机的 IP 与 MAC 地址

我们在电脑上向手机的地址 ping 一下,再输入 **arp -a** 指令查看 ARP 缓存,发现在我们电脑的 IP 地址 **10.130.60.25** 下并没有出现手机的 IP 地址。

```
S C:\WINDOWS\system32> ping 10.136.12.87

正在 Ping 10.136.12.87 具有 32 字节的数据:
来自 10.136.12.87 的回复: 字节=32 时间=375ms TTL=63
来自 10.136.12.87 的回复: 字节=32 时间=628ms TTL=63
来自 10.136.12.87 的回复: 字节=32 时间=1152ms TTL=63
来自 10.136.12.87 的回复: 字节=32 时间=197ms TTL=63

10.136.12.87 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 197ms, 最长 = 1152ms, 平均 = 588ms
S C:\WINDOWS\system32> arp -a

接口: 10.130.60.25 --- 0x4
Internet 地址      物理地址          类型
10.130.0.1          00-00-3e-00-01-fd 动态
10.130.127.255      ff-ff-ff-ff-ff-ff 静态
224.0.0.2           01-00-5e-00-00-02 静态
224.0.0.22          01-00-5e-00-00-16 静态
224.0.0.100         01-00-5e-00-00-64 静态
224.0.0.251         01-00-5e-00-00-fb 静态
224.0.0.252         01-00-5e-00-00-1c 静态
238.238.238.238     01-00-5e-0e-ee-ee 静态
239.238.237.236     01-00-5e-0e-ed-ac 静态
239.255.255.230     01-00-5e-1f-ff-fa 静态
255.255.255.255     ff-ff-ff-ff-ff-ff 静态

接口: 192.168.62.1 --- 0x10
Internet 地址      物理地址          类型
```

图 4.5: 电脑端查看 ARP 缓存

这说明电脑和手机不在同一广播域，这是因为校园网通常不是传统局域网（LAN），而是“大二层隔离”或“三层接入”网络。并且在校园网、公司网、公寓宽带等环境中，不同终端通常无法通过 ARP 互相发现，这是出于安全和管理考虑。因此我们需要使用一台外部手机热点的方式，将电脑和手机连接在同一个手机热点这个局域网下，从而来进行 ARP 请求与 ARP 响应。因为手机热点创建的是一个标准的 Wi-Fi 局域网（AP 模式），将电脑和手机都连接到同一个 NAT 网关（即手机）下，那么它们处于同一个广播域，可以互相发送/接收 ARP 广播包。

4.2 最终程序设计

4.2.1 引入头文件和库

```
1 #include <iostream>
2 #include <cstring>
3 #include <cstdint>
4 #include <winsock2.h>
5 #include <chrono>      // 用于精确计时
6 #include "pcap.h"      // Npcap 抓包库
7
8 #pragma comment(lib, "ws2_32.lib") // 链接 Windows 网络库
```

其中 `pcap.h` 和 `-lwpcap -lPacket` 是 Npcap 提供的抓包接口，允许我们发送/接收原始网络帧。

4.2.2 定义以太网帧和 ARP 报文结构

```
1 struct ethernet_header {
2     uint8_t dest_mac[6]; // 目标 MAC (ARP 请求填全 F)
3     uint8_t src_mac[6];  // 源 MAC (本机 MAC)
4     uint16_t ether_type;  // 上层协议类型 (0x0806 = ARP)
5 };
6
7 struct arp_header {
8     uint16_t htype; // 硬件类型 (1 = 以太网)
9     uint16_t ptype; // 协议类型 (0x0800 = IPv4)
10    uint8_t hlen;    // 硬件地址长度 (6 字节)
11    uint8_t plen;    // 协议地址长度 (4 字节)
12    uint16_t oper;   // 操作码 (1=请求, 2=响应)
```

```

13     uint8_t sender_mac[6]; // 发送方 MAC
14     uint8_t sender_ip[4];  // 发送方 IP
15     uint8_t target_mac[6]; // 目标 MAC (请求时填 0)
16     uint8_t target_ip[4];  // 目标 IP
17 };

```

按照 ARP 标准定义，确保构造的包能被其他设备识别。

4.2.3 定义全局变量，用于回调函数通信

```

1  uint8_t g_target_ip[4] = {0}; // 要查询的目标 IP
2  bool g_found = false;        // 是否找到
3  uint8_t g_result_mac[6] = {0}; // 找到的 MAC

```

我们这里选择使用全局变量，是因为我们在使用 `pcap_loop` 或 `pcap_next_ex` 的回调函数进行数据包的捕获时，不能直接传参，只能通过全局变量共享状态，因此我们通过全局变量的方式，定义好 `g_target_ip[4]` 和 `g_result_mac[6]`，来存储捕获数据包时的参数和得到 ARP 响应的结果。

4.2.4 数据包处理回调函数 packet_handler

```

1  void packet_handler(u_char* param, const struct pcap_pkthdr* header, const u_char*
    pkt_data) {
2      // 1. 检查包长度是否足够
3      if (header->caplen < sizeof(ethernet_header) + sizeof(arp_header)) return;
4      // 2. 解析以太网头
5      const ethernet_header* eth = reinterpret_cast<const ethernet_header*>(pkt_data);
6      if (ntohs(eth->ether_type) != 0x0806) return; // 不是 ARP 包
7      // 3. 解析 ARP 头
8      const arp_header* arp = reinterpret_cast<const arp_header*>(pkt_data +
        sizeof(ethernet_header));
9      if (ntohs(arp->oper) != 2) return; // 不是 ARP 响应
10     // 4. 检查是否是我们要找的 IP
11     if (memcmp(arp->sender_ip, g_target_ip, 4) == 0) {
12         memcpy(g_result_mac, arp->sender_mac, 6);
13         g_found = true;
14         std::cout << "\n 收到 ARP 响应! " << std::endl;
15         printf("IP: %d.%d.%d.%d --> MAC: %02X:%02X:%02X:%02X:%02X:%02X\n",
16             g_target_ip[0], ..., g_result_mac[5]);
17     }
18 }

```

该函数的关键点在于：

- `ntohs()`：将网络字节序转为主机字节序（因为网络数据是大端字节序）
- `memcmp()`：比较两个 IP 是否相同

4.2.5 IP 地址解析辅助函数 parse_ip

```

1 bool parse_ip(const std::string& ip_str, uint8_t* ip) {
2     int a, b, c, d;
3     if (sscanf(ip_str.c_str(), "%d.%d.%d.%d", &a, &b, &c, &d) != 4) return false;
4     // 检查范围 0~255
5     ip[0] = a; ip[1] = b; ip[2] = c; ip[3] = d;
6     return true;
7 }

```

将字符串 "192.168.136.215" 转为 4 字节数组 [192, 168, 136, 215]。

4.2.6 main 主函数

1: 初始化与设备选择

```

1 WSASStartup(...); // 启动 Windows 网络
2 pcap_findalldevs(...); // 列出所有网卡
3 // 用户选择设备 (如 Realtek 无线网卡)
4 pcap_open_live(...); // 打开设备, 进入混杂模式

```

这里的逻辑与我们之前的获取设备列表、选择并打开目标网卡设备的代码逻辑是一致的。

2: 输入本机信息和目标 IP 地址

我们需要输入：

- 本机 IP / MAC (用于填充 ARP 请求的"发送方"字段)
- 目标 IP (我们要查询的对象)

```

1 // 输入本机信息
2 std::string local_ip_str, local_mac_str;
3 uint8_t local_ip[4], local_mac[6];
4 std::cout << "\n请输入本机 IP 地址: ";
5 std::cin >> local_ip_str;
6 //调用parse_ip检查IP格式合法性
7 if (!parse_ip(local_ip_str, local_ip)) {
8     std::cout << "IP 格式错误! " << std::endl;
9     .....
10 }
11 std::cout << "请输入本机 MAC 地址 (aa:bb:cc:dd:ee:ff): ";
12 std::cin >> local_mac_str;
13 int mac[6];
14 if (sscanf(local_mac_str.c_str(), "%x:%x:%x:%x:%x:%x",
15           &mac[0], &mac[1], &mac[2], &mac[3], &mac[4], &mac[5]) != 6) {
16     std::cout << "MAC 格式错误! " << std::endl;
17     .....
18 }
19 for (int i = 0; i < 6; i++) {
20     if (mac[i] < 0 || mac[i] > 255) {

```

```

21         std::cout << "MAC 值非法!" << std::endl;
22         .....
23     }
24     local_mac[i] = static_cast<uint8_t>(mac[i]);
25 }
26 // 输入目标 IP
27 std::string target_ip_str;
28 std::cout << "请输入要查询的目标 IP 地址: ";
29 std::cin >> target_ip_str;
30 if (!parse_ip(target_ip_str, g_target_ip)) {
31     std::cout << "目标 IP 格式错误!" << std::endl;
32     .....
33 }

```

3: 构造 ARP 请求包

```

1  uint8_t packet[60] = {0}; // 以太网最小帧长 60 字节
2  // 填充以太网头
3  memset(eth->dest_mac, 0xFF, 6); // 广播地址
4  memcpy(eth->src_mac, local_mac, 6);
5  eth->ether_type = htons(0x0806); // ARP 类型
6  // 填充 ARP 头
7  arp->htype = htons(1);           // 以太网
8  arp->ptype = htons(0x0800);      // IPv4
9  arp->hlen = 6; arp->plen = 4;
10 arp->oper = htons(1);             // ARP 请求
11 memcpy(arp->sender_mac, local_mac, 6);
12 memcpy(arp->sender_ip, local_ip, 4);
13 memset(arp->target_mac, 0, 6); // 未知, 填 0
14 memcpy(arp->target_ip, g_target_ip, 4);

```

当这个构造的 ARP 请求包发出去后, 局域网所有设备都会收到, 只有目标 IP 会回复。

4: 发送 ARP 请求包, 并监听 ARP 响应包

```

1  pcap_sendpacket(handle, packet, 60); // 发送原始帧
2
3  // 使用 pcap_next_ex 循环监听, 最多 5 秒
4  auto start = std::chrono::steady_clock::now();
5  while (!g_found && (now - start < 5秒)) {
6      int res = pcap_next_ex(...);
7      if (res == 1) packet_handler(...); // 处理包
8  }

```

我们这里没有选择 pcap_loop 的方式, 而是选择 pcap_next_ex 函数的方式, 这样让我们能精确控制超时, 避免错过快速返回的 ARP Reply 响应数据包。

代码详见: [Network-technology-and-applications/实验三/src/arp_lookup_final.cpp](#)

5 程序运行结果

5.1 通过手机热点查看手机与电脑的 IP 与 MAC 地址

通过一个外部手机开启热点的方式，现在让我的手机和电脑都连接在这个热点上，然后我们通过这个外部手机的热点设置查看我们的手机和电脑的 IP 和 MAC 地址，如下所示：



图 5.6: 热点设置查看手机和电脑的 IP 与 MAC 地址

可以看到我的手机和电脑的 IP 与 MAC 地址如下所示：

设备	IP	MAC
电脑	192.168.136.215	F0:A6:54:8F:8D:D1
手机	192.168.136.217	9A:F4:43:09:88:77

表 1: 手机和电脑的 IP 与 MAC 地址

我们在电脑上通过命令行的方式向手机的 IP 进行 `ping` 命令,然后通过 `arp -a | findstr 192.168.136.217` 的方式，来进一步检验该 IP 地址与 MAC 地址的正确性。结果如下所示：

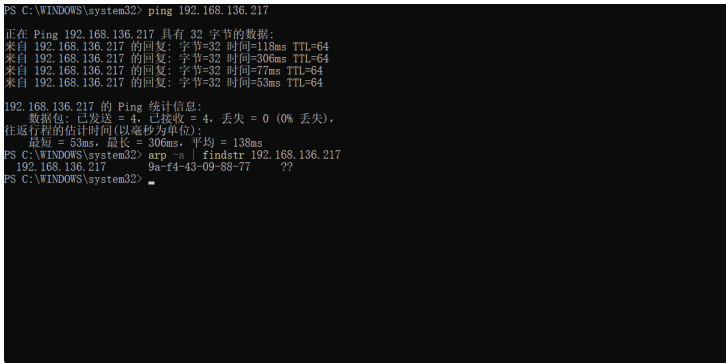


图 5.7: 电脑向手机 IP 发送 ping 命令并验证 MAC 地址

5.2 程序运行的 MAC 地址结果

我们采用如下命令进行编译运行：

编译运行指令

```
1 # 编译
2 g++ src/arp_lookup_final.cpp -o bin/arp_lookup_final.exe -I Include -L Lib/x64
   -lPacket -lwpcap -lws2_32 -liphlpapi -fexec-charset=UTF-8 -finput-charset=UTF-8
3 # 运行
4 ./bin/arp_lookup_final.exe
```

运行结果如下所示：

```
=== ARP-IP-MAC 映射查询工具（完整版） ===
1. \Device\NPF_{910F7DAB-50B3-4852-8704-2A74FC8A1661} - WAN Miniport (Network Monitor)
2. \Device\NPF_{6526F4B3-23F6-4839-BA2A-450E6A845412} - WAN Miniport (IPv6)
3. \Device\NPF_{22D86905-76CA-446A-8F57-DF5859B162FD} - WAN Miniport (IP)
4. \Device\NPF_{F3415588-79FA-42D8-B16D-9A6ED208D626} - Hyper-V Virtual Ethernet Adapter #2
5. \Device\NPF_{A61680AB-7DE1-443B-8DE7-1BF08098ECC6} - Hyper-V Virtual Ethernet Adapter
6. \Device\NPF_{0A887C94-9477-4075-8593-978E4211744E} - Realtek 8821CE Wireless LAN 802.11ac PCI-E NIC
7. \Device\NPF_{B114056E-1BB1-47FE-8230-EBC2A02752C4} - VMware Virtual Ethernet Adapter for VMnet8
8. \Device\NPF_{8005DF67-9681-404D-9710-DAF67A5A4025} - VMware Virtual Ethernet Adapter for VMnet1
9. \Device\NPF_{D907DE50-7F32-4366-B926-E7BC13B03530} - Microsoft Wi-Fi Direct Virtual Adapter #2
10. \Device\NPF_{56D1FF55-2E15-49DC-8D0D-0850383CA1FC} - Microsoft Wi-Fi Direct Virtual Adapter
11. \Device\NPF_{Loopback} - Adapter for loopback traffic capture
12. \Device\NPF_{143C9220-8B62-4871-A20C-823A8BF19746} - TAP-Windows Adapter V9
13. \Device\NPF_{C9BC77C9-69B2-462F-A2D8-EA367CAEDF68} - Wintun Userspace Tunnel
请选择设备编号 (1-13): 6

请输入本机 IP 地址: 192.168.136.215
请输入本机 MAC 地址 (aa:bb:cc:dd:ee:ff): F0:A6:54:8F:8D:D1
请输入要查询的目标 IP 地址: 192.168.136.217

正在发送 ARP 请求...
ARP 请求已发送, 等待响应 (最多 5 秒) ...

收到 ARP 响应!
IP: 192.168.136.217 --> MAC: 9A:F4:43:09:88:77
请按任意键继续. . .
PS D:\VS Code\NETWORK> █
```

图 5.8: 程序运行结果

可以看到我们程序对于手机 IP 192.168.136.217 所得到的 MAC 地址同样为 9A:F4:43:09:88:77，与我们手机热点中正确的手机 MAC 地址 9A:F4:43:09:88:77 是完全符合的，因此这也说明我们的程序正确得到了目标 IP 地址对应的 MAC 地址。